

AI for Productivity

Using AI Tools to Accelerate Technical Workflows

Dave Wade-Stein

COURSE OVERVIEW

What this course is...and isn't

- Practical ways to use AI in technical workflows: structuring prompts, iterating on outputs, and applying both to writing, debugging, summarization, and planning
- Not hype, but habits you can use in your daily work
- What you're permitted to put into these tools is a separate question— follow your organization's own policy

WHAT WE'LL COVER



How AI systems generate responses — enough depth to predict where they fail



Structuring prompts, and iterating when the first answer isn't right



Applying that to code, summarization, and repeatable workflows

WHAT WE WON'T



How the models work inside — architecture, training, or fine-tuning



Which tool or vendor to choose — these habits transfer across all of them



Building applications on AI APIs, agents, or automating at scale

Agenda at a glance

- 1 How AI Systems Work
- 2 Prompting as a Technical Skill
- 3 AI for Code and Technical Tasks
- 4 Summarization & Knowledge Extraction
- 5 Problem Solving and Iteration
- 6 Workflow Integration
- C Capstone: AI-Assisted Technical Task
- + Time Permitting: Intro to AI Agents



MODULE

How AI Systems Work (Mental Model)

OBJECTIVES

- Build a useful (not overly deep) mental model of AI systems
- Understand strengths and limitations in technical contexts

A working mental model

1

Input → pattern matching → output

AI predicts likely next words from patterns in training data, rather than looking facts up in a database.

2

Useful but imperfect

Outputs can be fluent and well-structured while still containing subtle errors.

3

Confident but wrong

The model's tone doesn't signal its accuracy — confident phrasing is not evidence of correctness.

4

Deterministic vs. non-deterministic

The same prompt can produce different outputs across runs — plan workflows that tolerate variation.

WHY THIS MATTERS

Treat AI as a search engine and surprises feel like bugs. Treat it as a pattern-completion engine and you know where to double-check: facts, numbers, edge cases, anything with one right answer. When the tool does search the web, the sources are real, but the summary written around them is still generated, so check both.

Generating vs. looking things up

NO RETRIEVAL

It generates

- The answer is built from statistical patterns in training data.
- Ask for sources and you may get plausible-looking citations — real-sounding authors and journals, titles that don't exist — delivered with the same confidence as real ones.

WITH RETRIEVAL

It looks things up

- When the tool searches, it fetches and reads real documents.
- The sources exist and the links work. This is a genuine lookup, and it removes the fabricated-citation problem.

EITHER WAY

You still verify

- Retrieval moves the risk. It doesn't remove it.
- The summary written around real sources is still generated — it can misread a claim, overstate a finding, or quietly drop a caveat. Confirming a citation exists isn't confirming it says what you were told.

Generating vs. looking things up (cont'd)

“What is happening when I ask you to provide a few examples of scholarly papers that discuss climate change—it's a lookup in that case, right?”

Two different things can happen, and the difference matters for your slide.

When I have search tools and use them — like the last several turns — it genuinely is a lookup. I issued queries, got back real documents, and read them. The citations in those answers point at pages that exist because I fetched them. That's retrieval, and the deck's claim doesn't describe it.

When I don't search, it's still generation, even when the output looks like a lookup. If you'd asked me for climate papers with no tools available, I'd have produced plausible-looking citations: real-sounding journals, real authors in the field, titles that fit the topic. Some would be real. Some would be composites — a real author, a real journal, a title that doesn't exist. And they'd all look equally confident. That's the classic fabricated-citation failure, and it comes from exactly the mechanism your slide describes.

The subtle part is that even the untooled version isn't purely made up. Training data contains a lot of text about climate papers, so there's real information encoded in the weights. But it's stored as statistical regularities, not as records. Recalling "a paper about X" is more like reconstructing a memory than reading a row from a table — and reconstruction fills gaps with whatever is plausible, without flagging where the seams are.

One caveat worth carrying into your course: retrieval doesn't fully solve this. I read real sources this session, but the sentences I wrote around them were still generated. That's where subtler errors live — attributing a claim to the wrong source, overstating what a document said, or smoothing over a hedge. Checking that a citation exists is not the same as checking that it says what the summary claims.

Exercise

EXERCISE**Test the model's explanation**

- Ask AI to explain a technical concept from your own work (a protocol, algorithm, library, or process).
- Read the explanation critically...does it match what you already know?
- Ask a pointed follow-up question designed to test correctness (e.g., an edge case or a “why” question).
- Note anywhere the second answer contradicts, qualifies, or adds nuance to the first.



MODULE

Prompting as a Technical Skill

OBJECTIVES

- Treat prompting as structured input design, not guesswork
- “Prompt engineering” vs. “Input design”

The four parts of a structured prompt



Context

What the AI needs to know: your environment, constraints, prior decisions.



Task

The specific action you want performed — stated as a clear instruction, not a question.



Constraints

Boundaries on the output: length, style, what to avoid, what to assume.



Format

How the output should be structured: bullets, table, code block, step list.

PRECISION VS. AMBIGUITY · ITERATION AS A WORKFLOW

Vague prompts force the model to guess at context, task, constraints, and format — and it will guess. Treat your first output as a draft: refine the prompt, don't just edit the result by hand.

Exercise

STEP 1

Convert vague prompts into structured ones

- Take 2–3 vague prompts—provided below, or your own—and rewrite each using Context / Task / Constraints / Forma.

NEED A STARTING POINT? TRY ONE OF THESE:

“Clean up my data”

“Summarize this climate report”

“Explain why the model results changed”

“Reword this paragraph”

“Finish this sentence for me”

“Make this email sound more professional”

STEP 2

Compare outputs and refine

- Run both the vague and structured versions. Compare the outputs side by side
- Refine the structured prompt once more based on what's still missing or unclear



MODULE

AI for Code and Technical Tasks

OBJECTIVES

- Use AI to assist with coding-related tasks, without losing ownership of the code

Four ways to bring AI into your code

1

Explaining code

Ask for a plain-language walkthrough of unfamiliar code — great for onboarding to a new codebase.

2

Generating small snippets

Use AI for well-scoped, low-risk code: boilerplate, utility functions, test scaffolding.

3

Debugging assistance

Share the error, the relevant code, and what you expected — not just “fix this.”

4

Refactoring suggestions

Ask for alternatives with trade-offs called out (readability, performance, safety) rather than a single “correct” rewrite.

A GOOD RULE OF THUMB

The less you'd trust a change from a junior teammate without review, the more scrutiny an AI-generated version needs too. AI accelerates typing, but it doesn't replace understanding the code you ship.

Exercise (alternate on next slide)

Working with a provided code sample, complete all three prompts below:

1. EXPLAIN

- Ask the AI to explain what the code does, line by line or section by section.
- Check the explanation against your own read of the code.

2. MODIFY

- Ask the AI to make a specific, well-scoped change (e.g., add error handling, change a data type).
- Verify the change didn't alter unrelated behavior.

3. FIND AN ISSUE

- Ask the AI to review the code for a bug, edge case, or risk.
- Confirm the issue is real before treating it as valid.

Exercise — alternate track

Working with the provided report instead of a code sample, complete all three prompts below:

1. EXPLAIN

- Ask the AI what the report argues, and what evidence it gives for each claim.
- Check that against your own read — does the evidence actually support the conclusion?

2. MODIFY

- Ask it to rewrite the recommendation so it claims only what the data supports.
- Check what it changed — and whether it softened a claim that should have been cut outright.

3. FIND AN ISSUE

- Ask it to review the report for errors, inconsistencies, or unsupported claims.
- Confirm each one yourself before accepting it. Some won't be real.



MODULE

Summarization and Knowledge Extraction

OBJECTIVES

- Process technical content efficiently without losing key detail

From wall of text to usable knowledge

1

Summarizing documentation

Condense long docs, RFCs, or specs into a scannable overview.

2

Extracting key ideas

Pull out decisions, risks, or requirements buried in dense text.

3

Turning text into steps or checklists

Convert narrative instructions into an ordered, actionable list.

4

Asking targeted follow-ups

Drill into one section instead of re-summarizing the whole document.

DEMO

Live: article → summary → checklist

- Summarize a technical article, then extracts it into a set of actionable steps — watch how the follow-up prompt narrows the ask.

Exercise

STEP 1**Summarize the provided content**

- Using the technical content provided, ask the AI for a concise summary aimed at someone who hasn't read the original.

STEP 2**Convert to a checklist or steps**

- Ask the AI to restructure the summary into a numbered checklist or step-by-step list.
- Check it against the source — did anything important get dropped or reordered?



MODULE

Problem Solving and Iteration

OBJECTIVES

- Use AI as a structured problem-solving tool, not a one-shot oracle

Ask → evaluate → refine

1

Break it down

Split a large or vague problem into smaller, addressable parts.

2

Ask

Prompt for a first attempt at the smallest useful piece.

3

Evaluate

Check the response against what you actually need.

4

Refine

Adjust the prompt — add constraints, correct assumptions, ask alternatives.

DEMO

Live: refining a vague problem

- Start with a deliberately vague problem statement and refines it through several prompt iterations — note how each round adds a constraint.

Exercise

EXERCISE**Solve a small problem through iteration**

- Choose a small, real problem from your own work (a query, a config, a small script, a process).
- Solve it using at least 2–3 prompt iterations — don't accept the first answer as final.
- At each round, write down what you changed in the prompt and why.
- Ask AI to propose at least one alternative approach before you settle on a solution.



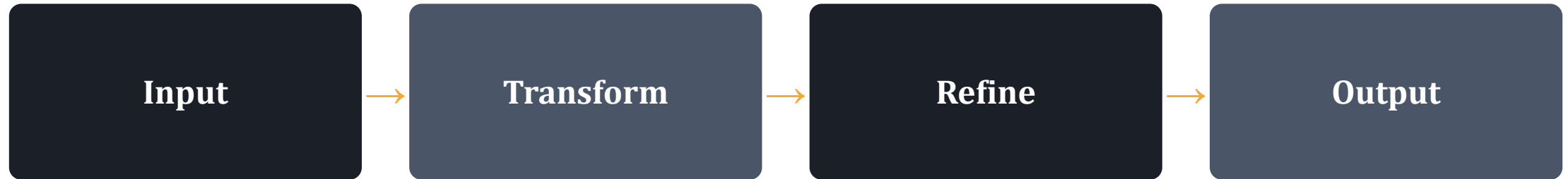
MODULE

Workflow Integration

OBJECTIVES

- Integrate AI into repeatable, multi-step workflows

A repeatable four-stage workflow



1

When to use AI vs. not

Reach for AI when a task is well-scoped and verifiable — skip it when correctness depends on context only you have.

2

Verifying outputs

Every workflow needs an explicit check step — verification is not optional overhead, it's part of the workflow.

DEMO

Example workflow

- Describe problem → generate solution → refine constraints → validate output.

Exercise

Pick one track and sketch out your own Input → Transform → Refine → Output workflow:

TRACK A

Coding task

- Example: turning a bug report into a verified fix.
- Define what “input” is (the report), how AI transforms it, what you refine, and how you'll validate the output before merging.

TRACK B

Research task

- Example: turning a pile of sources into a decision brief.
- Define what “input” is (the sources), how AI transforms it, what you refine, and how you'll validate the output before sharing it.



CAPSTONE

AI-Assisted Technical Task

Objective: apply AI to complete a structured, multi-step technical task — choosing from one of three scenarios.

REQUIREMENTS

≥ 3

Use at least three prompts to complete the task

1+

Refine at least one response based on evaluation



Validate the output — don't blindly trust it

Choose your scenario

SCENARIO 1

Debug & document

- You're handed a small, broken script with no comments.
- Use AI to diagnose the bug, propose a fix, and write a short explanation of the root cause for a teammate.

SCENARIO 2

Research brief

- You're given several pages of technical material on an unfamiliar topic.
- Use AI to summarize it, extract the key decisions or risks, and produce a one-page brief.

SCENARIO 3

Build a mini workflow

- You have a repetitive multi-step task (e.g., formatting a report, triaging tickets).
- Design and test an Input → Transform → Refine → Output workflow for it.

A Brief Introduction to AI Agents

Not covered in the live session — included as reference. A high-level look at AI agents and how they extend prompting into automated, multi-step workflows.

PROMPTING vs. AGENTS, IN ONE LINE

Prompting = one request → one response

Agents = goal → multiple steps → evolving responses

An agent is a system that uses an AI model to make decisions, performs actions (often via tools or code), and repeats that loop until it reaches a goal.

How agents differ from basic prompting

Basic Prompting	AI Agents
One input → one output	Multi-step process
User drives each step	System can act autonomously
No memory, by default	May track state or history
Static interaction	Iterative and dynamic

Agents are not “smarter” — they are more structured and more automated uses of the same underlying models.

Example, where they're used, and the catch

CONCEPTUAL EXAMPLE

Instead of a single prompt — “Summarize this document” — an agent might, all automatically:

- Read the document
- Identify key sections
- Summarize each section
- Combine into a final summary
- Refine the result

WHERE AGENTS ARE USED

- Automating repetitive workflows
- Multi-step research tasks
- Code generation and iteration
- Data processing pipelines

IMPORTANT CAVEAT

Agents can compound errors if not designed carefully. Validation becomes even more important — more automation $\neq$ more correctness.

WHAT TO LEARN NEXT

1

Multi-step prompting techniques

2

Tool use (APIs, code execution)

3

Frameworks for building agents

4

Designing and validating workflows

Make it a habit, not a one-off

- AI is a probabilistic pattern-completion tool — useful, imperfect, and worth verifying
- Structured prompts (Context, Task, Constraints, Format) beat vague ones
- Treat every AI output as a first draft: ask, evaluate, refine
- Build workflows — Input → Transform → Refine → Output — you can reuse
- Validation is part of the task, not an optional extra step

THIS WEEK, TRY:

- Rewrite one prompt using C/T/Co/F
- Run a 2-step iteration on a real task
- Build one repeatable workflow